

# Modeling and Simulation of a Two-Link Planar Robotic Arm using SolidWorks and MATLAB Simscape Multibody

LUCKSHAN S.M.

Department of Mechanical Engineering  
University of Moratuwa  
Sri Lanka  
luckshansm.23@uom.lk

**Abstract**—This paper presents the modeling and simulation of a two-link planar robotic arm. The mechanical model of the arm is designed in SolidWorks and exported into MATLAB using Simscape Multibody. In Simulink, the arm is placed in a closed-loop control system that consists of a desired trajectory generator, an inverse kinematics block, a PID controller for each joint, and a forward kinematics block that is used to plot the actual end-effector path. The desired trajectory is a circular loop of radius 0.3 centered at  $(1, -1)$ , and both links have a length of 1. The PID controller was tuned using the PID Tuner app, which reduced the overshoot of the step response from 24.6 % to 4.13 % and gave a settling time of 0.00121 seconds. The results are presented as XY plots of the desired and the actual end-effector paths.

**Index Terms**—two-link planar robotic arm, SolidWorks, Simscape Multibody, Simulink, inverse kinematics, forward kinematics, PID control

## I. INTRODUCTION

This paper documents a mechatronics project on a two-link planar robotic arm. The arm is first modeled in SolidWorks and then exported into MATLAB using Simscape Multibody, where it is simulated together with a control system built in Simulink. The workflow of modeling the arm in SolidWorks and exporting it to Simscape Multibody follows the tutorial by MT Engineering [1].

The key contributions of this work include:

- Design of a two-link planar robotic arm in SolidWorks and its import into Simscape Multibody
- Implementation in Simulink of a desired trajectory generator, inverse kinematics, one PID controller per joint, and forward kinematics
- Tuning of the PID controller using the PID Tuner app and comparison of the step response before and after tuning
- Plotting of the desired and the actual end-effector paths

## II. MECHANICAL MODEL

### A. Arm Design in SolidWorks

The arm was designed in SolidWorks as an assembly named Assem1. As shown in Fig. 1(a), it consists of a circular base and two links. The two rotational joints of the arm are de-

scribed by the joint angles  $\theta_1$  and  $\theta_2$ .

### B. Export to Simscape Multibody

The SolidWorks assembly was exported for use in MATLAB. The MATLAB current folder in Fig. 1(b) shows the exported files: Assem1.xml, three part files (Part1, Part2 and Part3 in STEP format), and Assem1\_DataFile.m. The Simulink model Assem1.slx is saved in the same folder. The arm is visualized in Mechanics Explorer, where the model appears as Assem1 with a subsystem named Robot\_Mechanical\_Model.

## III. CONTROL SYSTEM DESIGN

### A. Overall Simulink Model

Fig. 2 shows the Simulink model. A clock block provides the time  $u$  to the Desired\_Trajectory block, which outputs the desired end-effector position  $(x_d, y_d)$ . The inverse\_kinematics block converts this position into the desired joint angles  $\theta_{1d}$  and  $\theta_{2d}$ . For each joint, the desired angle is compared with the joint angle coming from the Robot Mechanical Model at a summing junction, and the error is passed to a PID controller whose output drives the corresponding input of the Robot Mechanical Model. The joint angles of the Robot Mechanical Model are also fed to the Forward\_Kinematics block, which outputs the actual end-effector position  $(x_a, y_a)$ . The desired path is plotted in XY Graph2 and the actual path in XY Graph1. The simulation stop time is 10.0 and the solver is set to auto (ode45).

### B. Desired Trajectory

The desired trajectory is a circular loop defined in the Desired\_Trajectory block (Fig. 3) as

$$x_d = x_c + r \cos(\omega u) \quad (1)$$

$$y_d = y_c - r \sin(\omega u) \quad (2)$$

with the workspace center  $(x_c, y_c) = (1, -1)$ , the loop radius  $r = 0.3$ , and  $\omega = 2\pi$ , which gives one full loop per unit time.

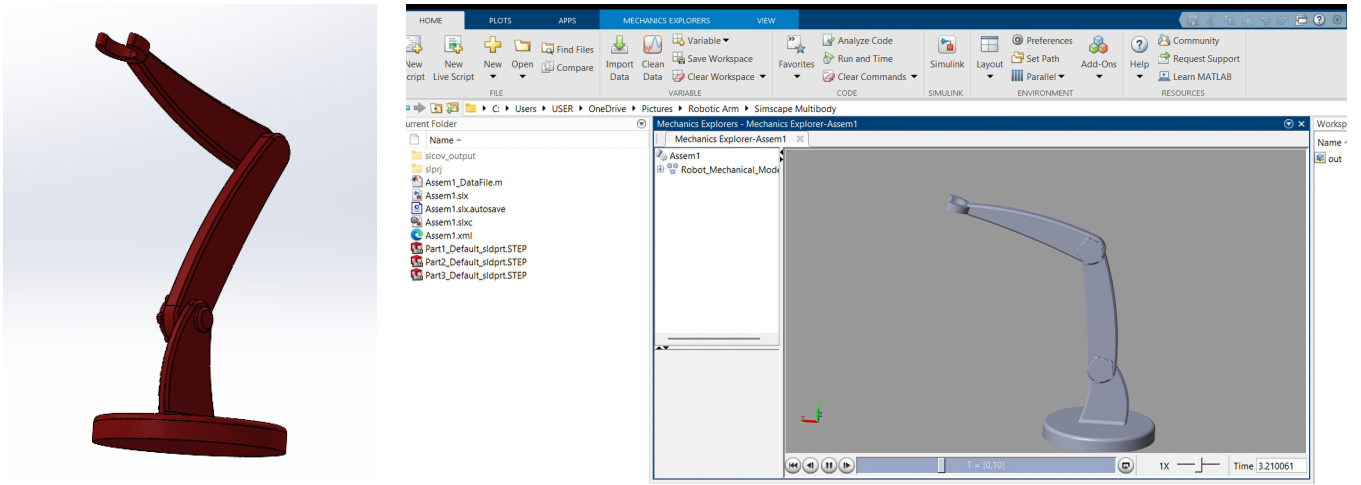


Fig. 1. (a) SolidWorks model of the two-link planar robotic arm. (b) The arm in Simscape Multibody Mechanics Explorer, with the exported files shown in the MATLAB current folder.

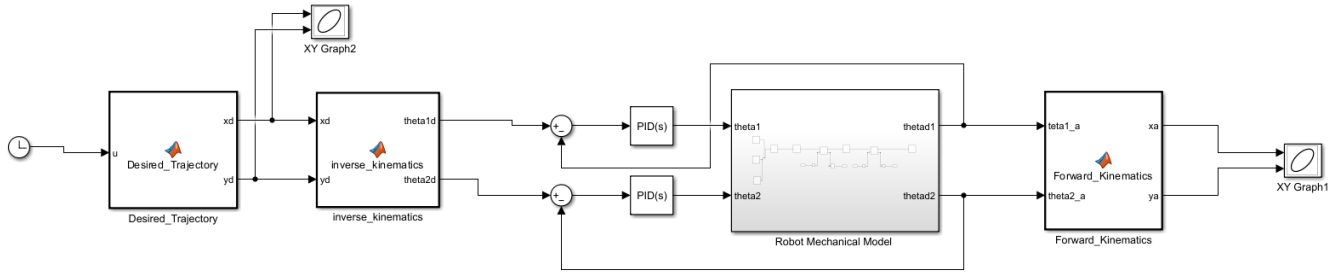


Fig. 2. Simulink model of the closed-loop control system.

```

1 function [xd,yd] = Desired_Trajectory(u)
2 xc = 1; % workspace center x
3 yc = -1; % workspace center y
4 r = 0.3; % loop radius
5 w = 2*pi; % one full loop per unit time
6
7 xd = xc + r*cos(w*u);
8 yd = yc - r*sin(w*u);
9 end

```

Fig. 3. Code of the Desired\_Trajectory block.

```

1 function [theta1d,theta2d] = inverse_kinematics(xd,yd)
2 l1 = 1;
3 l2 = 1;
4
5 cosTheta2 = (xd^2 + yd^2 - l1^2 - l2^2) / (2*l1*l2);
6 cosTheta2 = max(min(cosTheta2, 1), -1); % guard against rounding
7
8 theta2d = acos(cosTheta2);
9 theta1d = atan2(yd, xd) - atan2(l2*sin(theta2d), l1 + l2*cos(theta2d));
10 end

```

Fig. 4. Code of the inverse\_kinematics block.

### C. Inverse Kinematics

The `inverse_kinematics` block (Fig. 4) uses the link lengths  $l_1 = l_2 = 1$  and computes the desired joint angles as

$$\cos \theta_2 = \frac{x_d^2 + y_d^2 - l_1^2 - l_2^2}{2l_1l_2} \quad (3)$$

$$\theta_{2d} = \arccos(\cos \theta_2) \quad (4)$$

$$\theta_{1d} = \text{atan2}(y_d, x_d) - \text{atan2}(l_2 \sin \theta_{2d}, l_1 + l_2 \cos \theta_{2d}) \quad (5)$$

The value of  $\cos \theta_2$  is limited to the range  $[-1, 1]$  in the code to guard against rounding.

### D. PID Controller and Tuning

Each joint is controlled by a PID Controller block of the form

$$P + I \frac{1}{s} + D \frac{N}{1 + N \frac{1}{s}} \quad (6)$$

with a filtered derivative. The controller parameters are set from an internal source, and the tuning method is Trans-

fer Function Based (PID Tuner app). The tuned values shown in the block parameters (Fig. 5) are  $P = 17.8447$ ,  $I = 10562.9861$ ,  $D = 0.0065535$  and  $N = 768834.4422$ .

Fig. 6 shows the reference tracking step response given by the PID Tuner, where the solid line is the tuned response and the dashed line is the response with the parameters that were in the block. Table I compares the two.

### E. Forward Kinematics

The `Forward_Kinematics` block takes the actual joint angles of the Robot Mechanical Model as inputs and gives the actual end-effector position  $(x_a, y_a)$  as outputs. This position is plotted in XY Graph1 and compared with the desired path in XY Graph2.

## IV. SIMULATION RESULTS

Fig. 7 shows the desired trajectory from XY Graph2. The loop spans  $x$  from 0.7 to 1.3 and  $y$  from  $-0.7$  to  $-1.3$ , which

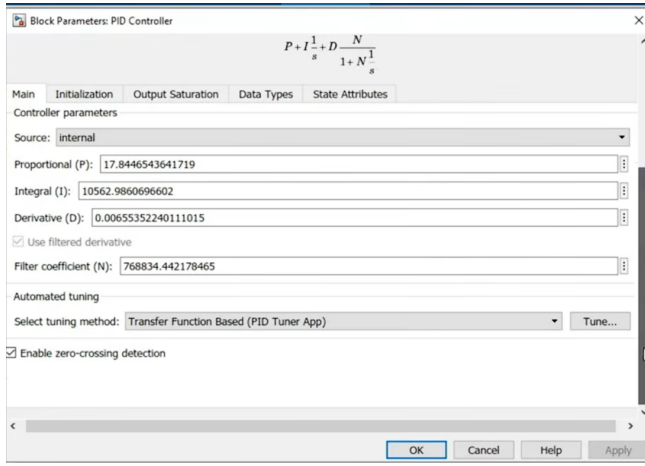


Fig. 5. Block parameters of the PID Controller.

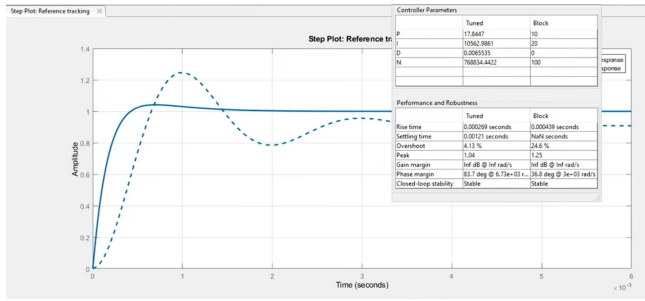


Fig. 6. Step plot of the reference tracking in the PID Tuner.

TABLE I  
TUNED AND BLOCK CONTROLLER PARAMETERS AND PERFORMANCE

|                       | Tuned       | Block      |
|-----------------------|-------------|------------|
| P                     | 17.8447     | 10         |
| I                     | 10562.9861  | 20         |
| D                     | 0.0065535   | 0          |
| N                     | 768834.4422 | 100        |
| Rise time             | 0.000269 s  | 0.000439 s |
| Settling time         | 0.00121 s   | NaN        |
| Overshoot             | 4.13 %      | 24.6 %     |
| Peak                  | 1.04        | 1.25       |
| Gain margin           | Inf dB      | Inf dB     |
| Phase margin          | 83.7 deg    | 36.8 deg   |
| Closed-loop stability | Stable      | Stable     |

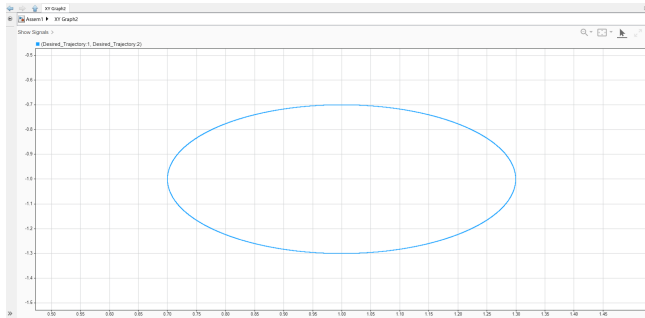


Fig. 7. Desired end-effector trajectory (XY Graph2).

agrees with the center  $(1, -1)$  and the radius  $r = 0.3$  set in the Desired\_Trajectory block. The axes of the plot have different scales, so the circle appears as an ellipse.

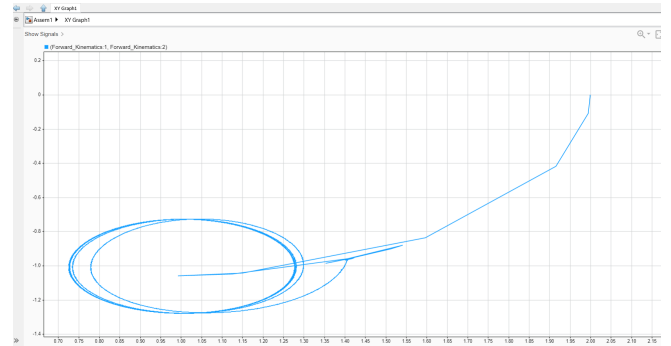


Fig. 8. Actual end-effector path from the forward kinematics (XY Graph1).

Fig. 8 shows the actual end-effector path from XY Graph1. The path starts at approximately  $(2.0, 0)$ , which is the full reach of the arm ( $l_1 + l_2 = 2$ ). It then moves toward the desired loop and continues in looping paths of about the same size and position as the desired circle, spanning approximately  $x = 0.75$  to  $1.3$  and  $y = -0.73$  to  $-1.28$ . Small differences between the successive loops and the desired path can be seen in the plot.

## V. CONCLUSION

A two-link planar robotic arm was modeled in SolidWorks, exported into MATLAB using Simscape Multibody, and simulated in Simulink with a closed-loop control system. The system uses a circular desired trajectory, inverse kinematics to obtain the joint angles, a PID controller at each joint, and forward kinematics to obtain the actual end-effector path. The PID Tuner app gave an overshoot of 4.13 % and a settling time of 0.00121 seconds, compared with an overshoot of 24.6 % for the parameters that were previously in the block. The actual end-effector path follows the region of the desired circular loop after an initial movement from the starting position, with small differences from the desired path visible in the plot.

## REFERENCES

- [1] MT Engineering, "The Full Modeling and simulation of a Robotic Arm using MATLAB simscape multibody and Solidworks," YouTube, May 2022. [Online]. Available: <https://youtu.be/pDiwAA1cnb0>